Ejercicio 7. Exploración y gestión de metadatos con File

Enunciado:

Desarrollar un programa en Java que utilice la clase File para interactuar con la ruta "datos/prueba.txt": comprobar si existe, crearlo si no existe controlando la IOException, y mostrar por consola el nombre (getName()) y la ruta (getPath()) del archivo.

Explicación de la solución:

Se construye un objeto File con la ruta datos/prueba.txt. Como createNewFile() falla si la carpeta datos no existe, antes se crea con mkdirs() a partir de getParentFile(). Luego exists() decide si el archivo se crea o ya estaba disponible; la IOException de createNewFile() se captura con try-catch. Finalmente, tanto si existía como si se acaba de crear, se muestran el último nombre de la ruta con getName() y la ruta con la que se construyó el objeto con getPath().

Código fuente (Ejercicio7ExplorarArchivo.java):

```
import java.io.File;
import java.io.IOException;

public class Ejercicio7ExplorarArchivo {
    private static final String RUTA = "datos/prueba.txt";

    public static void main(String[] args) {
        File archivo = new File(RUTA);

        // La carpeta "datos" debe existir; si no, createNewFile() lanza IOException
        File carpeta = archivo.getParentFile();
        if (carpeta != null && !carpeta.exists()) {
            carpeta.mkdirs();
        }

        if (!archivo.exists()) {
            try {
                archivo.createNewFile();
                System.out.println("[OK] El archivo no existia y fue creado.");
            } catch (IOException ex) {
                System.out.println("[ERROR] No fue posible crear el archivo: " + ex.getMessage());
            }
        } else {
            System.out.println("[AVISO] El archivo ya existia.");
        }

        System.out.println("Nombre: " + archivo.getName());
        System.out.println("Ruta:   " + archivo.getPath());
    }
}
```

Resultado de la ejecución:

```
--- Primera ejecución ---
[OK] El archivo no existia y fue creado.
Nombre: prueba.txt
Ruta:   datos/prueba.txt

--- Segunda ejecución ---
[AVISO] El archivo ya existia.
Nombre: prueba.txt
Ruta:   datos/prueba.txt
```

Ejercicio 8. Escritura estructurada por líneas con BufferedWriter

Enunciado:

Desarrollar un programa en Java que cree un fichero de texto llamado materias.txt usando un BufferedWriter enlazado con un FileWriter, escriba cinco asignaturas (una por línea con newLine()), cierre el flujo con close() y controle la IOException.

Explicación de la solución:

Se configura un BufferedWriter sobre un FileWriter con la ruta materias.txt. Un arreglo guarda las cinco asignaturas y un ciclo for-each las escribe con write(); después de cada una se invoca newLine(), que inserta el salto de línea propio del sistema operativo, de modo que queda exactamente una asignatura por línea. Los errores de escritura se capturan con IOException y el flujo se cierra con close() dentro del bloque finally, para que se libere aunque ocurra un error.

Código fuente (Ejercicio8EscribirMaterias.java):

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class Ejercicio8EscribirMaterias {
    private static final String RUTA = "materias.txt";

    public static void main(String[] args) {
        String[] materias = {
            "Programacion Orientada a Objetos",
            "Bases de Datos",
            "Estructuras de Datos",
            "Redes de Computadoras",
            "Sistemas Operativos"
        };

        BufferedWriter salida = null;
        try {
            salida = new BufferedWriter(new FileWriter(RUTA));
            for (String materia : materias) {
                salida.write(materia);
                salida.newLine();
            }
            System.out.println("[OK] Se escribieron " + materias.length
                    + " asignaturas en " + RUTA);
        } catch (IOException ex) {
            System.out.println("[ERROR] No se pudo escribir en el archivo: " + ex.getMessage());
        } finally {
            try {
                if (salida != null) {
                    salida.close();
                }
            } catch (IOException ex) {
                System.out.println("[ERROR] No se pudo cerrar el archivo: " + ex.getMessage());
            }
        }
    }
}
```

Resultado de la ejecución:

```
[OK] Se escribieron 5 asignaturas en materias.txt

Contenido de materias.txt:
Programacion Orientada a Objetos
Bases de Datos
Estructuras de Datos
Redes de Computadoras
Sistemas Operativos
```

Ejercicio 9. Control del modo de apertura (append vs sobrescribir)

Enunciado:

A partir del archivo materias.txt del ejercicio anterior, desarrollar un programa que lo abra con FileWriter en modo de adición (append = true), añada dos nuevas asignaturas al final usando newLine() y cierre el flujo con close(). Verificar que se conserven las cinco asignaturas originales.

Explicación de la solución:

El segundo parámetro del constructor de FileWriter controla el modo de apertura: con false (valor por defecto) el archivo se vacía y se escribe desde cero, mientras que con true los datos nuevos se agregan al final y el contenido previo se conserva. Por eso se usa new FileWriter(RUTA, true). Se escriben dos asignaturas más, cada una seguida de newLine() para no romper el formato por líneas, y el flujo se cierra con close() en el bloque finally.

Código fuente (Ejercicio9AgregarMaterias.java):

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class Ejercicio9AgregarMaterias {
    private static final String RUTA = "materias.txt";

    public static void main(String[] args) {
        BufferedWriter salida = null;
        try {
            // El segundo parametro (true) activa el modo "append"
            salida = new BufferedWriter(new FileWriter(RUTA, true));
            salida.write("Ingles Tecnico");
            salida.newLine();
            salida.write("Ingenieria de Software");
            salida.newLine();
            System.out.println("[OK] Se anadieron 2 asignaturas al final de " + RUTA);
        } catch (IOException ex) {
            System.out.println("[ERROR] No se pudo escribir en el archivo: " + ex.getMessage());
        } finally {
            try {
                if (salida != null) {
                    salida.close();
                }
            } catch (IOException ex) {
                System.out.println("[ERROR] No se pudo cerrar el archivo: " + ex.getMessage());
            }
        }
    }
}
```

Resultado de la ejecución:

```
[OK] Se anadieron 2 asignaturas al final de materias.txt

Contenido de materias.txt:
Programacion Orientada a Objetos
Bases de Datos
Estructuras de Datos
Redes de Computadoras
Sistemas Operativos
Ingles Tecnico
Ingenieria de Software
```

Ejercicio 10. Ciclo de vida completo y borrado seguro

Enunciado:

Desarrollar un programa en Java que represente la ruta "datos/temporal.txt" con un objeto File, consulte su estado con exists() y lo cree si no existe controlando la IOException, declare una variable booleana confirmarBorrado para simular la confirmación del usuario y use delete() solo si es verdadera, imprimiendo el resultado.

Explicación de la solución:

Se crea el objeto File con datos/temporal.txt (asegurando antes que exista la carpeta datos) y se consulta con exists(); si no está, se crea con createNewFile() dentro de un try-catch. Luego la variable confirmarBorrado simula la pregunta de confirmación al usuario. El método delete() se ejecuta exclusivamente cuando esa variable es true, y el valor booleano que devuelve se imprime en consola. Si fuera false, el programa solo avisa que el borrado fue cancelado.

Código fuente (Ejercicio10CicloVida.java):

```
import java.io.File;
import java.io.IOException;

public class Ejercicio10CicloVida {
    private static final String RUTA = "datos/temporal.txt";

    public static void main(String[] args) {
        File archivo = new File(RUTA);

        File carpeta = archivo.getParentFile();
        if (carpeta != null && !carpeta.exists()) {
            carpeta.mkdirs();
        }

        // Consulta del estado y creacion segura
        if (!archivo.exists()) {
            try {
                archivo.createNewFile();
                System.out.println("[OK] Archivo creado: " + archivo.getPath());
            } catch (IOException ex) {
                System.out.println("[ERROR] No fue posible crear el archivo: " + ex.getMessage());
            }
        } else {
            System.out.println("[AVISO] El archivo ya existe: " + archivo.getPath());
        }

        // Simula la confirmacion del usuario antes de borrar
        boolean confirmarBorrado = true;

        if (confirmarBorrado) {
            boolean eliminado = archivo.delete();
```

```
            System.out.println("Resultado de delete(): " + eliminado);
        } else {
            System.out.println("[AVISO] Borrado cancelado por el usuario.");
        }
    }
}
```

Resultado de la ejecución:

```
--- Primera ejecución ---
[OK] Archivo creado: datos/temporal.txt
Resultado de delete(): true

--- Segunda ejecución ---
[OK] Archivo creado: datos/temporal.txt
Resultado de delete(): true
```

Conclusiones

1. La clase File permite comprobar, crear, consultar y eliminar archivos con métodos como exists(), createNewFile(), getName(), getPath() y delete(); además, es necesario asegurar que la carpeta contenedora exista antes de crear el archivo.

2. BufferedWriter junto con newLine() permite escribir texto estructurado en una línea por registro de forma portable entre sistemas operativos, y close() garantiza que los datos queden guardados en disco.

3. El segundo parámetro de FileWriter define si se sobrescribe (false) o se añade al final (true), por lo que es clave para no perder información previa del archivo.

4. Validar con exists() y una confirmación booleana antes de delete(), y capturar la IOException, hace que el programa sea más seguro y entregue mensajes claros ante cualquier situación.